

AS61B – Fundamentos de Programação

Métodos de Ordenação

Matheus Fernando Mollon

07/05/2024

Introdução

- O problema de ordenação de elementos consiste em:
- Permutar ou rearranjar elementos de um vetor $v[0..n-1]$, onde n é o número total de elementos do vetor, de tal modo que eles fiquem ordenados:

$$v[0] \leq v[1] \leq v[2] \leq \dots \leq v[n-1] \text{ (ordem crescente)}$$

ou

$$v[0] \geq v[1] \geq v[2] \geq \dots \geq v[n-1] \text{ (ordem decrescente)}$$

Bubble Sort

Bubble sort (método da Bolha)

- O método *bubble sort* é o algoritmo mais simples de ordenação que executa repetidamente a troca de elementos adjacentes se estiverem em ordem errada.
- Esse método é altamente iterativo, percorrendo todo o vetor n vezes.
 - A cada passada o **maior** elemento é “flutuado” ao **final** do vetor (em caso de ordenação **crescente**).

Simulação

<https://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html>

```
// Bubble sort 1
#include <stdio.h>
#define tam 5

int main(){
    //Declaração de variáveis
    int vet[5];
    int i,j,aux;

    //Entrada de dados
    for(i = 0; i < tam; i++){
        printf("Digite o valor %d: ",i+1);
        scanf("%d",&vet[i]);
    }
}
```

```
//Bubble sort
for(i = 0; i<tam; i++){
    for(j = 1; j < tam; j++){
        if(vet[j-1] > vet[j]){
            aux = vet[j];
            vet[j] = vet[j-1];
            vet[j-1] = aux;
        }
    }
}
```

```
//Saída de dados
printf("\n\nVetor ordenado:\n");
for(i = 0; i < tam; i++){
    printf("%d ", vet[i]);
}
}
```

```
return 0;
```

Bubble sort 1 – versão mais simples

Simulação do algoritmo Bubble sort

Iteração 1 ($i = 0$)

	0	1	2	3	4
	5	2	8	1	7
$j = 1$	2	5	8	1	7
$j = 2$	2	5	8	1	7
$j = 3$	2	5	1	8	7
$j = 4$	2	5	1	7	8

Simulação do algoritmo Bubble sort

Iteração 1 ($i = 0$)

	0	1	2	3	4
	5	2	8	1	7
$j = 1$	2	5	8	1	7
$j = 2$	2	5	8	1	7
$j = 3$	2	5	1	8	7
$j = 4$	2	5	1	7	8

Iteração 2 ($i = 1$)

	0	1	2	3	4
	2	5	1	7	8
$j = 1$	2	5	1	7	8
$j = 2$	2	1	5	7	8
$j = 3$	2	1	5	7	8
$j = 4$	2	1	5	7	8

Simulação do algoritmo Bubble sort

Iteração 3 ($i = 2$)

	0	1	2	3	4
	2	1	5	7	8
$j = 1$	1	2	5	7	8
$j = 2$	1	2	5	7	8
$j = 3$	1	2	5	7	8
$j = 4$	1	2	5	7	8

Simulação do algoritmo Bubble sort

Iteração 3 ($i = 2$)

	0	1	2	3	4
	2	1	5	7	8
$j = 1$	1	2	5	7	8
$j = 2$	1	2	5	7	8
$j = 3$	1	2	5	7	8
$j = 4$	1	2	5	7	8

Iteração 4 ($i = 3$)

	0	1	2	3	4
	1	2	5	7	8
$j = 1$	1	2	5	7	8
$j = 2$	1	2	5	7	8
$j = 3$	1	2	5	7	8
$j = 4$	1	2	5	7	8

Simulação do algoritmo Bubble sort

Iteração 5 ($i = 4$)

Final

	0	1	2	3	4
	1	2	5	7	8
$j = 1$	1	2	5	7	8
$j = 2$	1	2	5	7	8
$j = 3$	1	2	5	7	8
$j = 4$	1	2	5	7	8

1	2	5	7	8
---	---	---	---	---

Resumo:

20 comparações

5 Trocas

Simulação do algoritmo Bubble sort

Iteração 5 ($i = 4$)

	0	1	2	3	4
	1	2	5	7	8
$j = 1$	1	2	5	7	8
$j = 2$	1	2	5	7	8
$j = 3$	1	2	5	7	8
$j = 4$	1	2	5	7	8

Considere “*tam*” o tamanho do vetor. É necessário repetir o método “*tam*” vezes para se ter o vetor ordenado?

1ª Otimização:

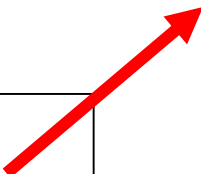
```
//Bubble sort
for(i = 0; i < tam-1; i++){
    for(j = 1; j < tam; j++){
        if(vet[j-1] > vet[j]){
            aux = vet[j];
            vet[j] = vet[j-1];
            vet[j-1] = aux;
        }
    }
}
```

Simulação do algoritmo Bubble sort

Considere “*tam*” o tamanho do vetor. É necessário repetir o método “*tam*” vezes para se ter o vetor ordenado?

1ª Otimização:

```
//Bubble sort
for(i = 0; i < tam-1; i++){
    for(j = 1; j < tam; j++){
        if(vet[j-1] > vet[j]){
            aux = vet[j];
            vet[j] = vet[j-1];
            vet[j-1] = aux;
        }
    }
}
```



Como podemos otimizar ainda mais este método?

Vamos a um segundo exemplo de simulação do Bubble Sort:

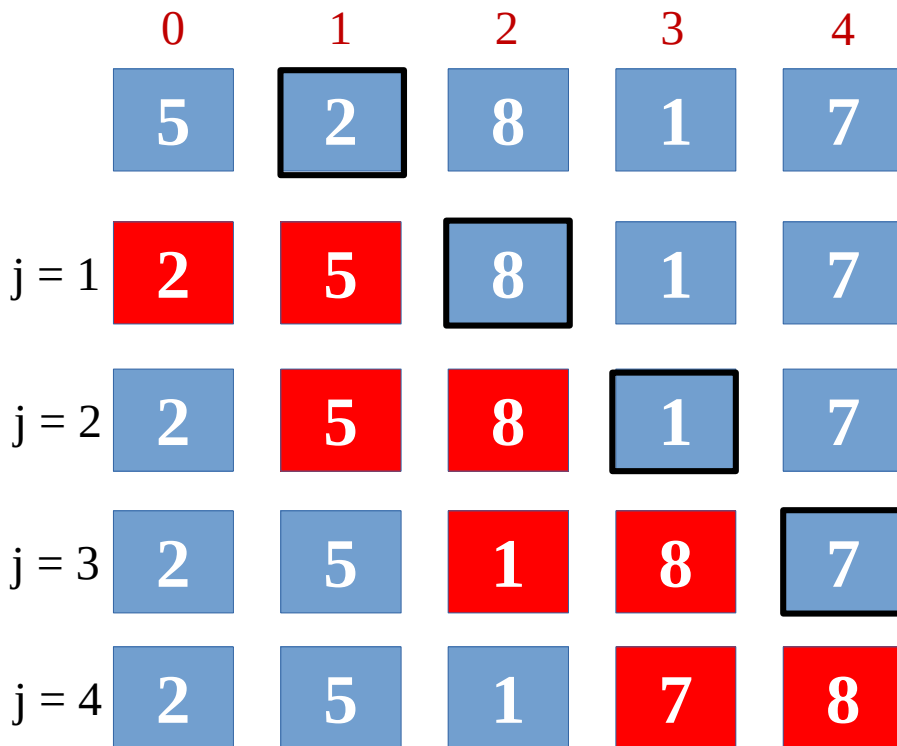
Simulação do algoritmo Bubble sort

Iteração 1 ($i = 0$)

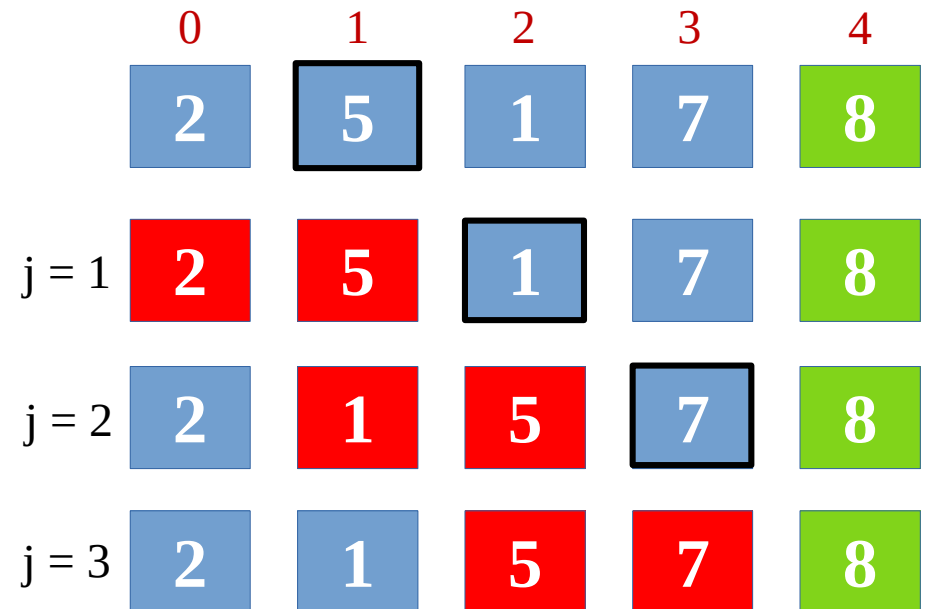
	0	1	2	3	4
	5	2	8	1	7
$j = 1$	2	5	8	1	7
$j = 2$	2	5	8	1	7
$j = 3$	2	5	1	8	7
$j = 4$	2	5	1	7	8

Simulação do algoritmo Bubble sort

Iteração 1 ($i = 0$)



Iteração 2 ($i = 1$)



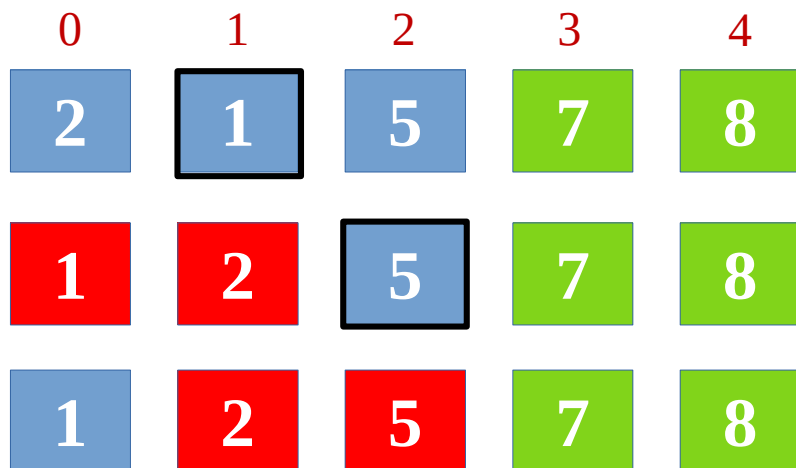
Simulação do algoritmo Bubble sort

Iteração 3 ($i = 2$)

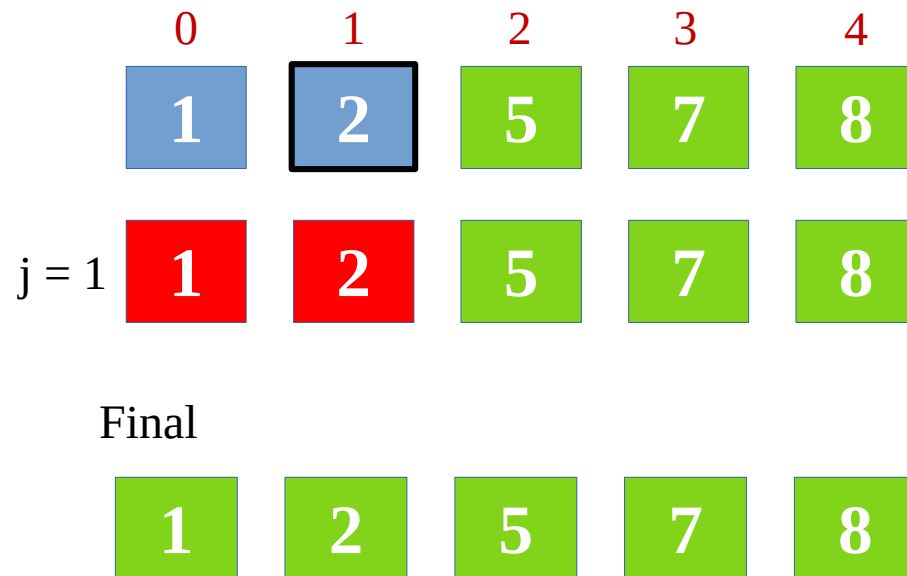
	0	1	2	3	4
	2	1	5	7	8
$j = 1$	1	2	5	7	8
$j = 2$	1	2	5	7	8

Simulação do algoritmo Bubble sort

Iteração 3 ($i = 2$)



Iteração 4 ($i = 3$)



Resumo:

10 comparações

5 Trocas

Simulação do algoritmo Bubble sort

Iteração 4 ($i = 3$)



A cada iteração do laço externo, é necessário percorrer **totalmente** o vetor realizando as comparações entre os elementos adjacentes?

$j = 1$



Final



Simulação do algoritmo Bubble sort

A cada iteração do laço externo, é necessário percorrer **totalmente** o vetor realizando as comparações entre os elementos adjacentes?



Não é necessário, pois a cada repetição do método o maior ou menor elemento é flutuado para o final do vetor, sendo posicionado corretamente.

2ª Otimização:

```
//Bubble sort
for(i = 0; i<tam-1; i++){
    for(j = 1; j < tam - i; j++){
        if(vet[j-1] > vet[j]){
            aux = vet[j];
            vet[j] = vet[j-1];
            vet[j-1] = aux;
        }
    }
}
```



Logo, a cada iteração do método, pode-se desconsiderar os elementos que já estão corretamente posicionados, evitando-se realizar comparações desnecessárias.

Simulação do algoritmo Bubble sort

A cada iteração do laço externo, é necessário percorrer **totalmente** o vetor realizando as comparações entre os elementos adjacentes?



Não é necessário, pois a cada repetição do método o maior ou menor elemento é flutuado para o final do vetor, sendo posicionado corretamente.

2ª Otimização:

```
//Bubble sort
for(i = 0; i<tam-1; i++){
    for(j = 1; j < tam - i; j++){
        if(vet[j-1] > vet[j]){
            aux = vet[j];
            vet[j] = vet[j-1];
            vet[j-1] = aux;
        }
    }
}
```



Logo, a cada iteração do método, pode-se desconsiderar os elementos que já estão corretamente posicionados, evitando-se realizar comparações desnecessárias.

Como atividade, encontre outras maneiras para otimizar ainda mais este método.

```

// Bubble sort 2
#include <stdio.h>
#define tam 5

int main(){
    //Declaração de variáveis
    int vet[5];
    int i,j,aux;

    //Entrada de dados
    for(i = 0; i < tam; i++){
        printf("Digite o valor %d: ",i+1);
        scanf("%d",&vet[i]);
    }

    //Bubble sort
    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }

    //Saída de dados
    printf("\n\nVetor ordenado:\n");
    for(i = 0; i < tam; i++){
        printf("%d ", vet[i]);
    }

    return 0;
}

```

Bubble sort 2 – versão otimizada

Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	0
i	0
j	

	0	1	2	3	4
vet	5	2	8	1	7

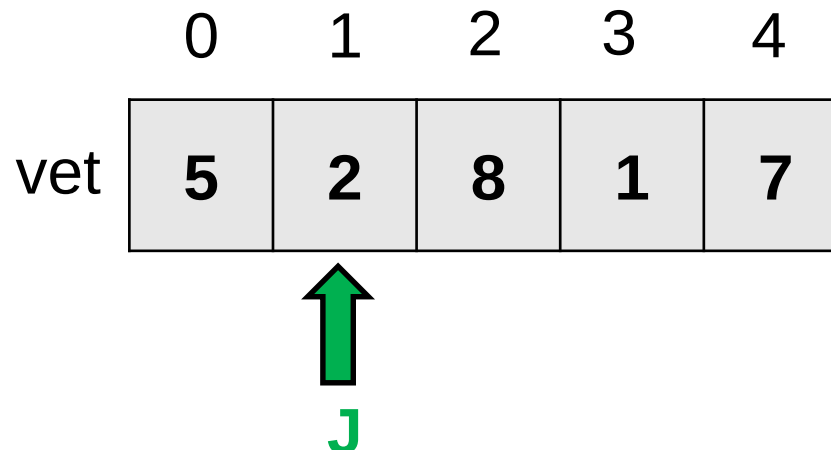
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
         for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	0
i	0
j	1



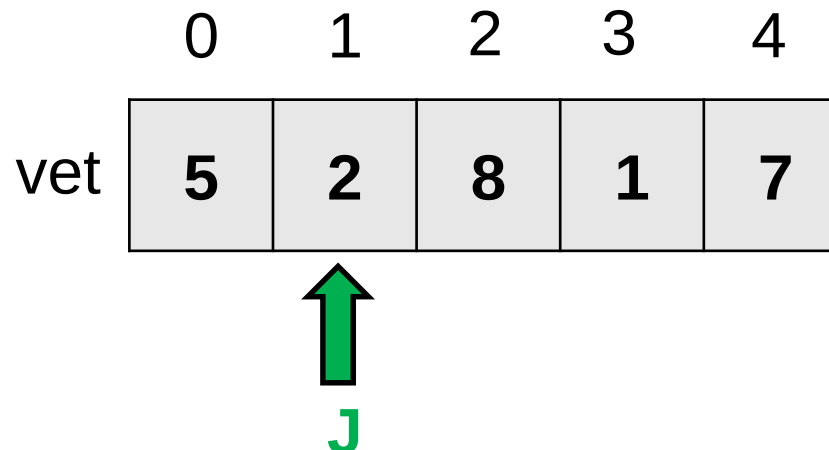
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	0
i	0
j	1



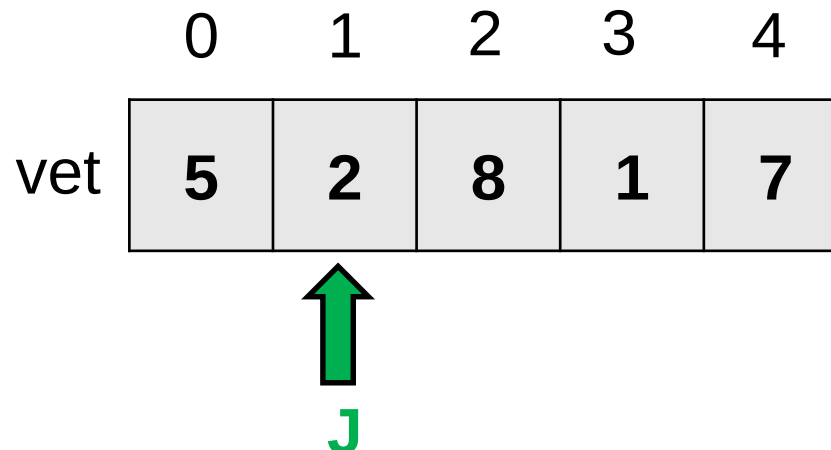
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	0
i	0
j	1



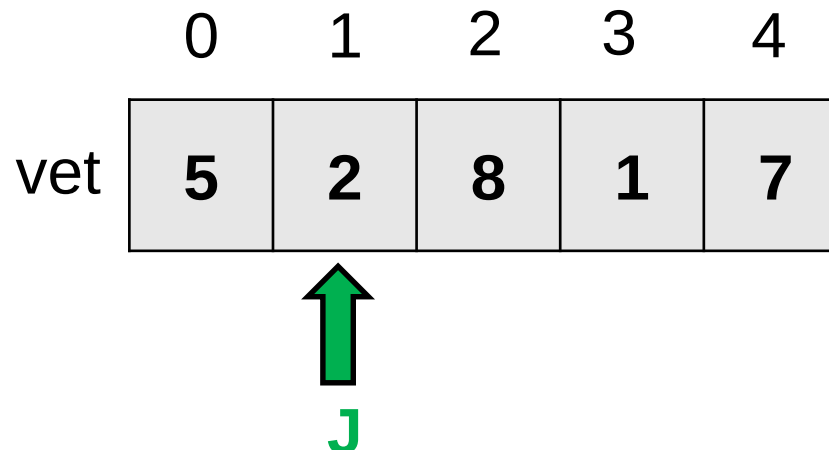
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	2
i	0
j	1



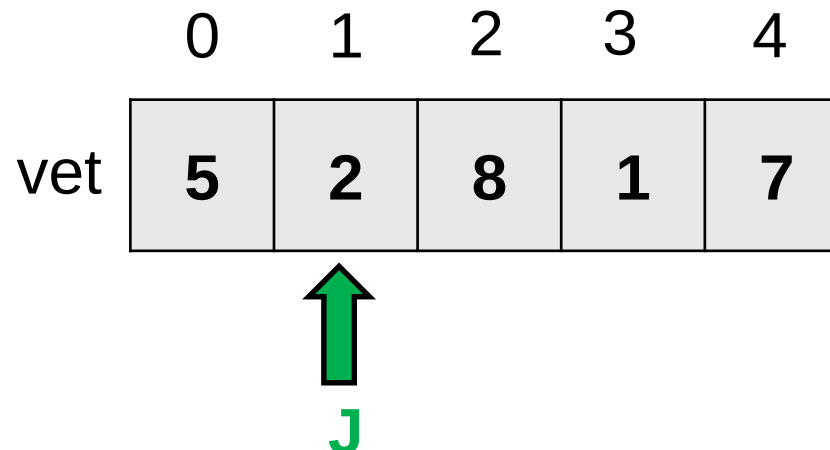
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	2
i	0
j	1



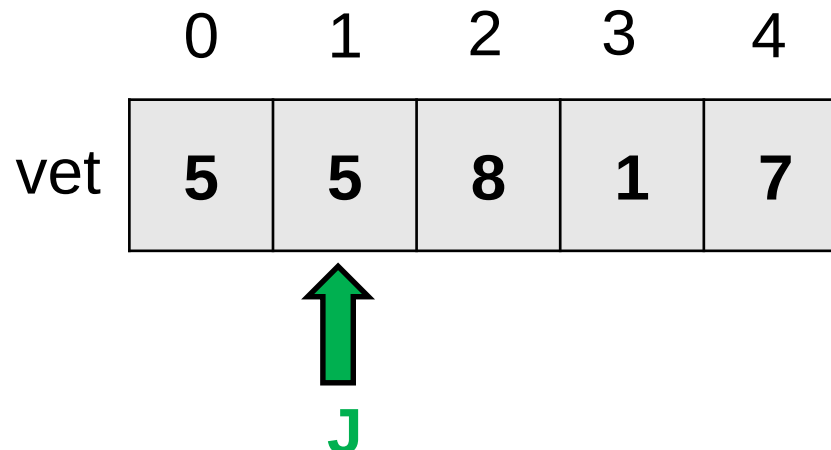
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	2
i	0
j	1



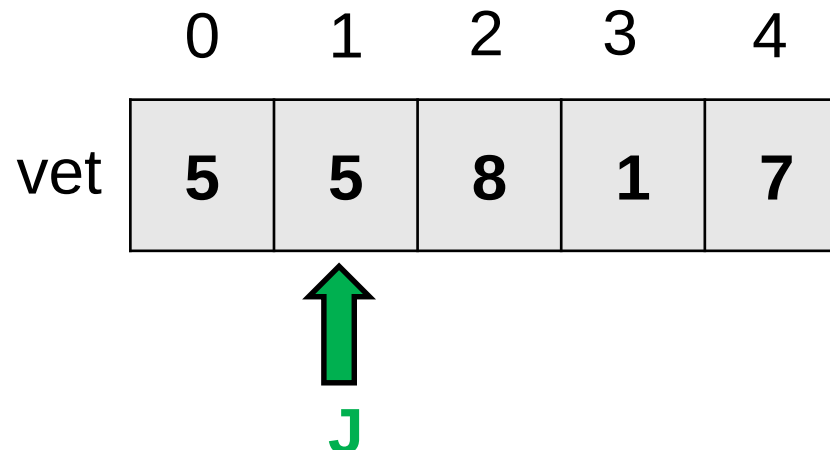
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	2
i	0
j	1



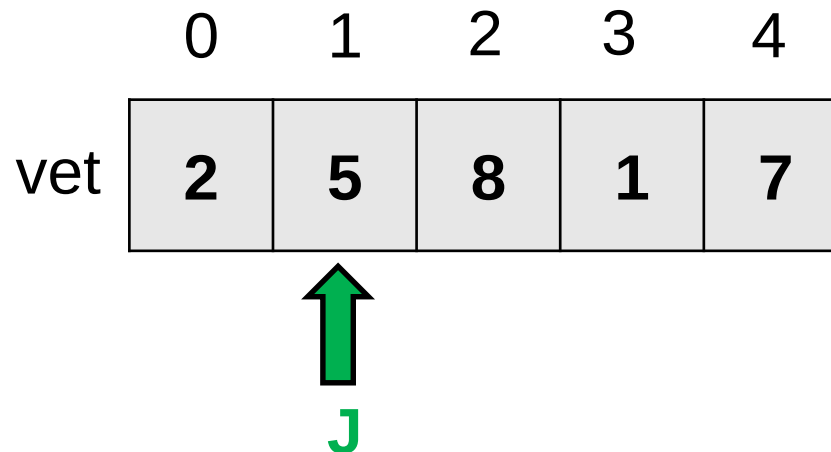
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	2
i	0
j	1



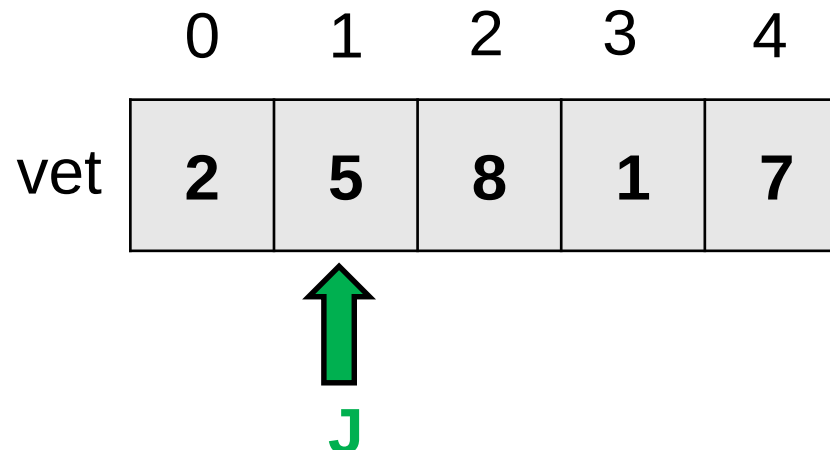
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
         for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	2
i	0
j	1



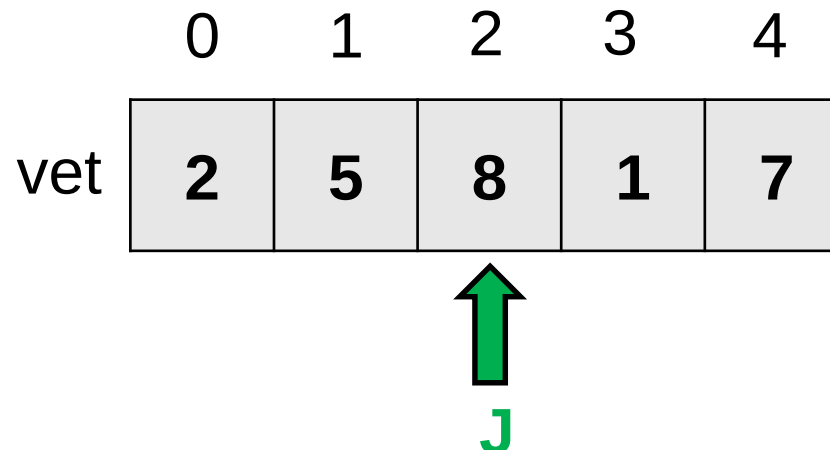
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
         for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	2
i	0
j	2



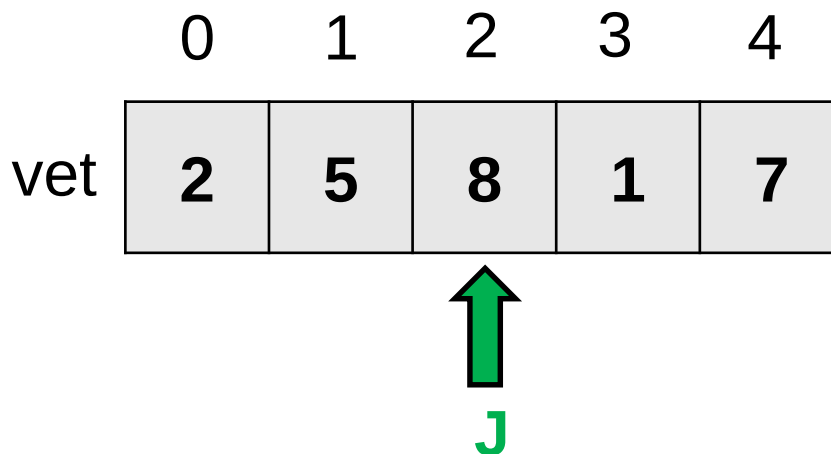
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	2
i	0
j	2



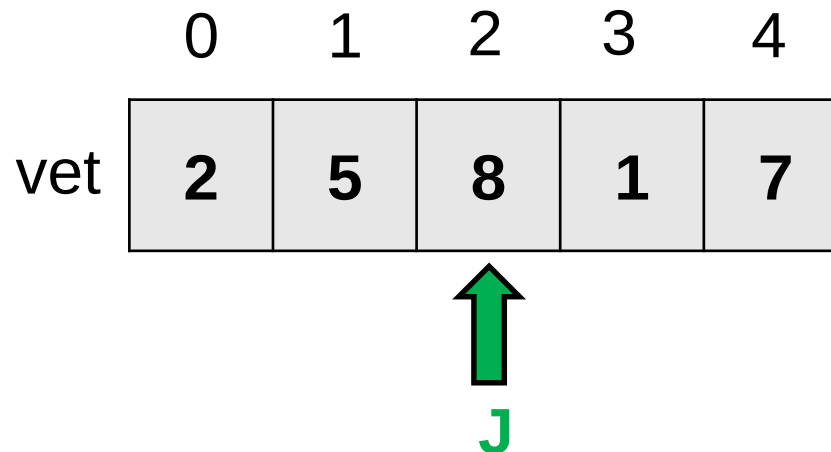
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	2
i	0
j	2



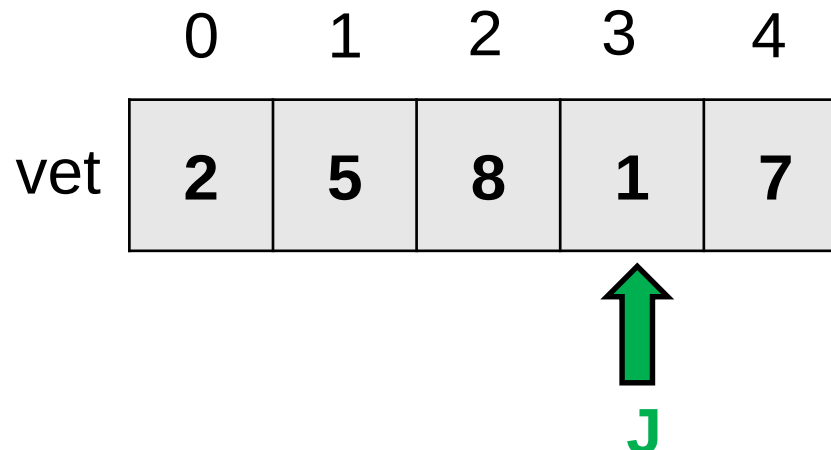
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	2
i	0
j	3



Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

8 > 1 (V)

Variável	Conteúdo
aux	2
i	0
j	3

	0	1	2	3	4
vet	2	5	8	1	7

J

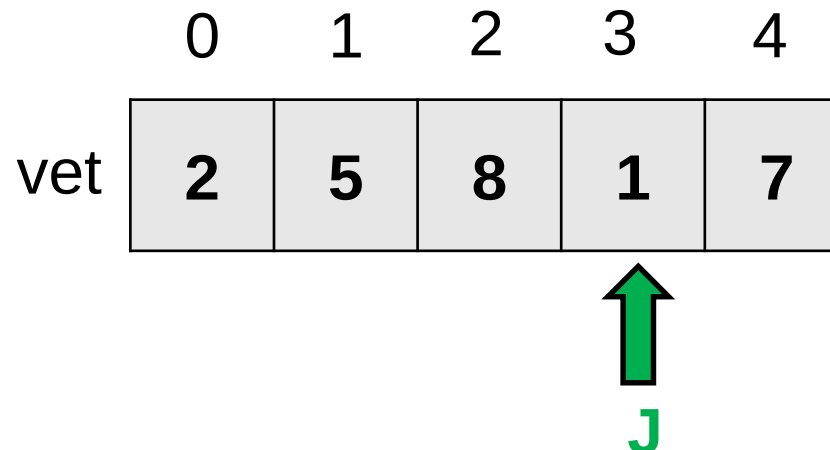
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	2
i	0
j	3



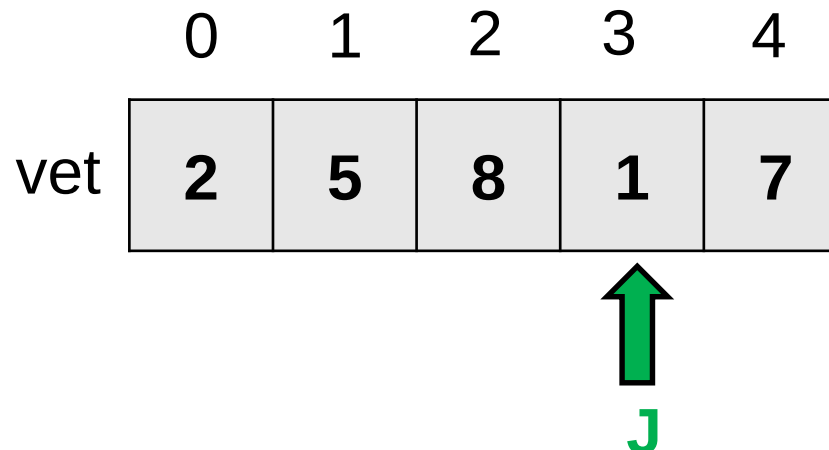
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	0
j	3



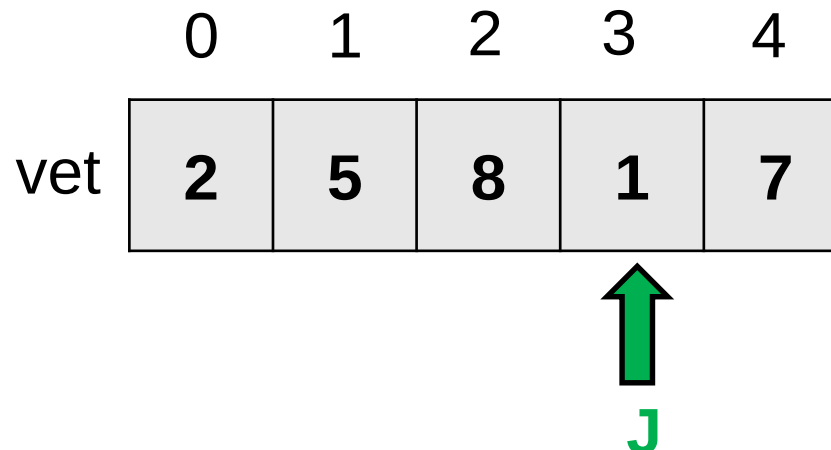
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	0
j	3



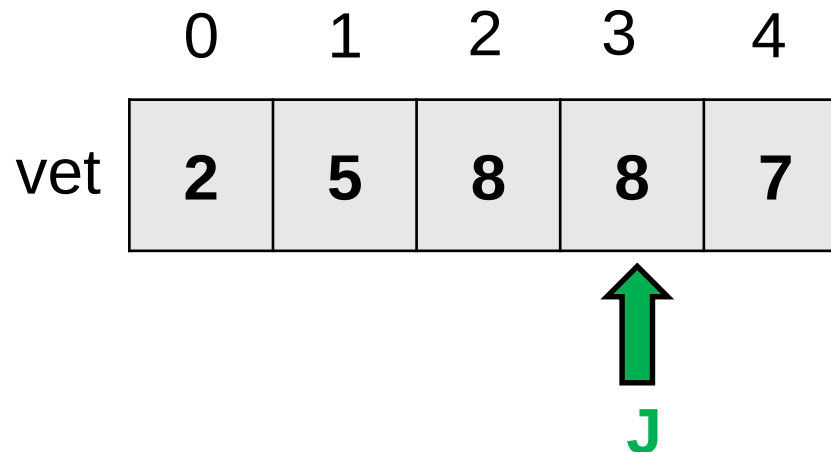
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	0
j	3



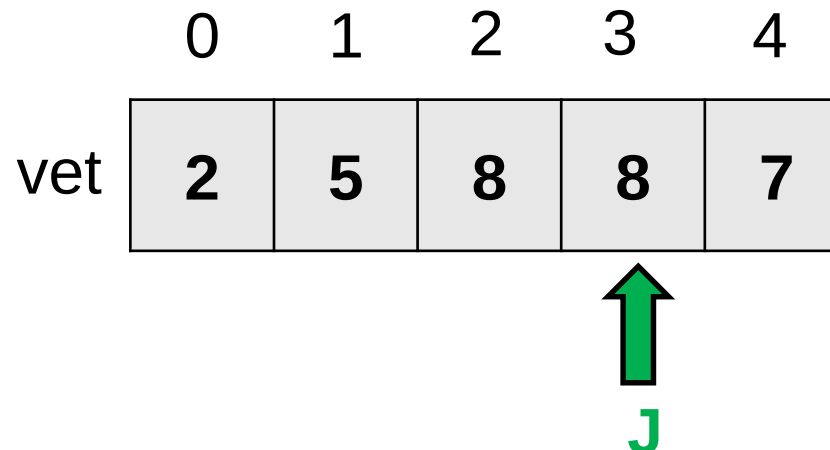
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	0
j	3



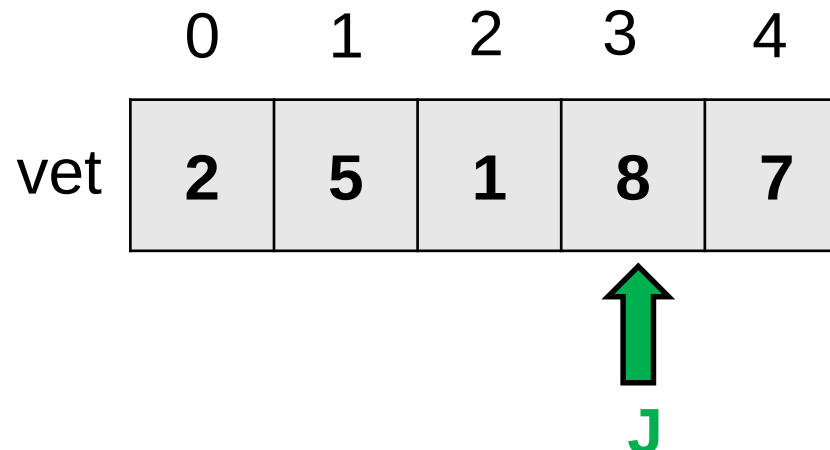
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	0
j	3



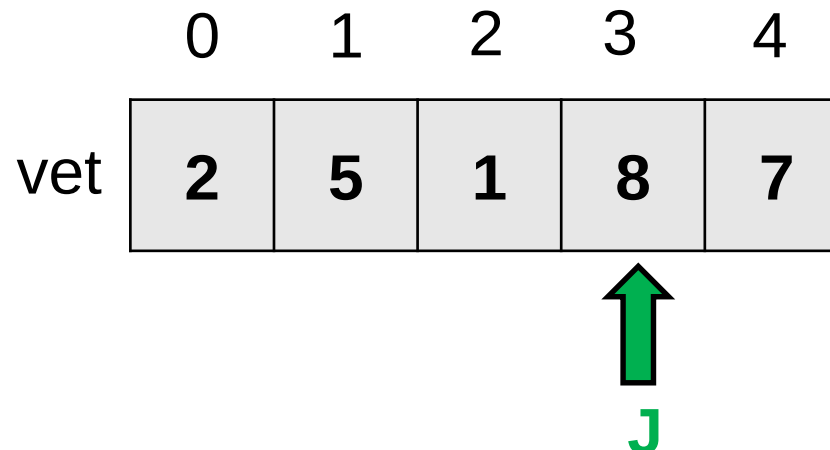
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
         for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	0
j	3



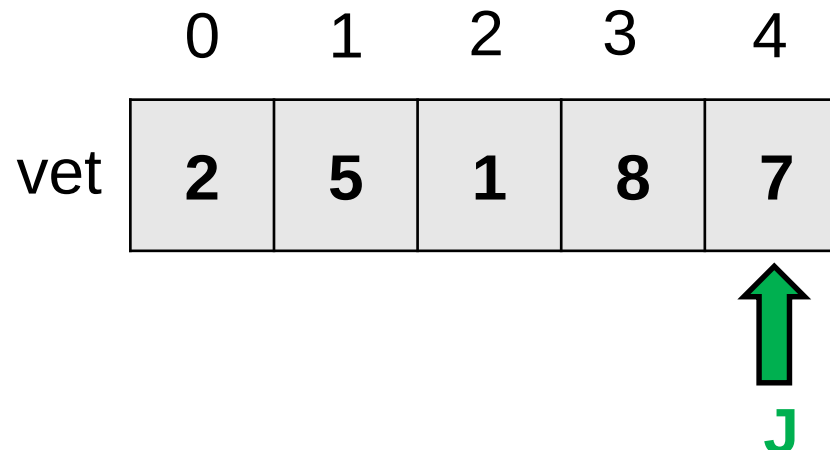
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
         for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	0
j	4



Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

8 > 7 (V)

Variável	Conteúdo
aux	1
i	0
j	4

	0	1	2	3	4
vet	2	5	1	8	7

↑
j

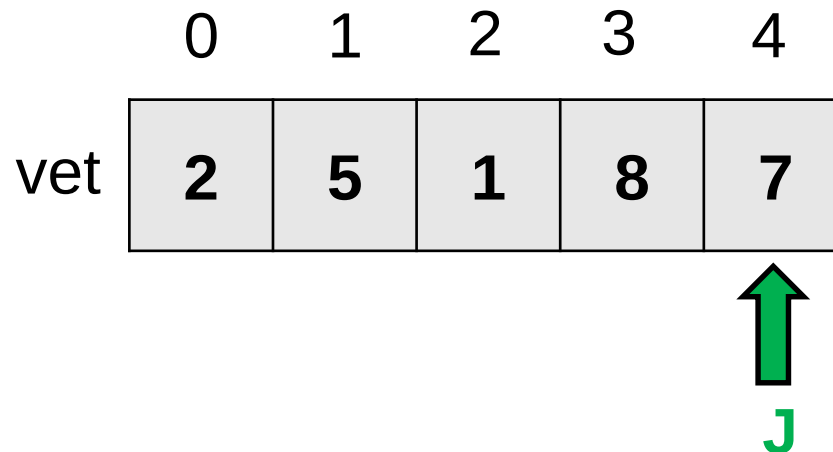
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	0
j	4



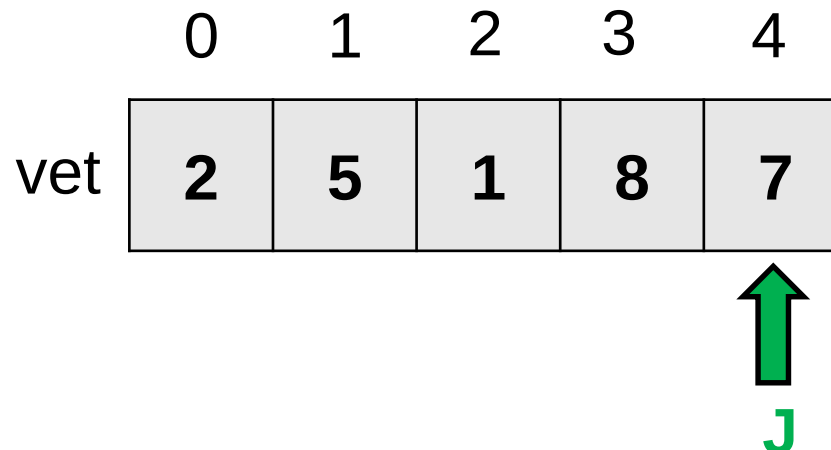
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	7
i	0
j	4



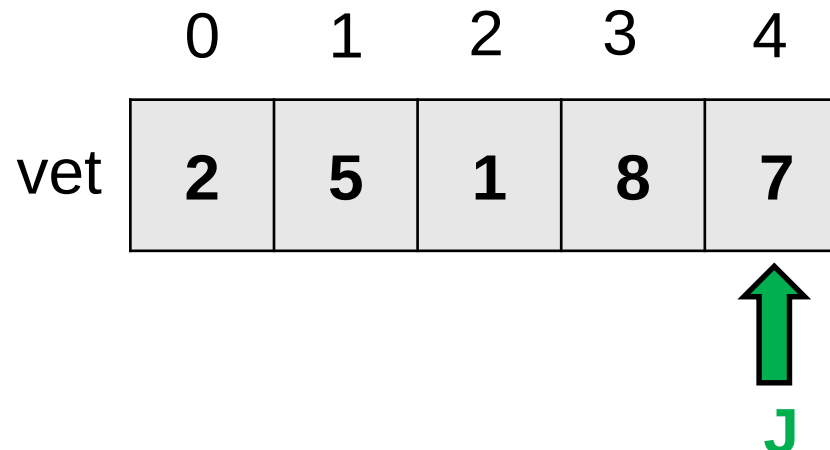
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	7
i	0
j	4



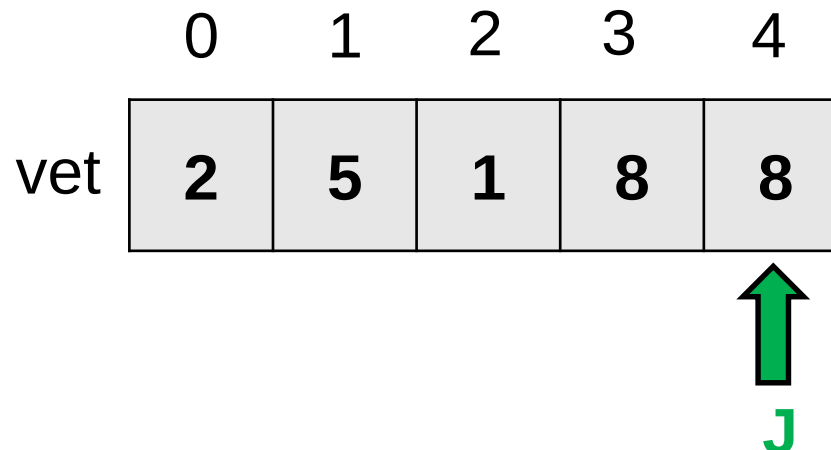
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	7
i	0
j	4



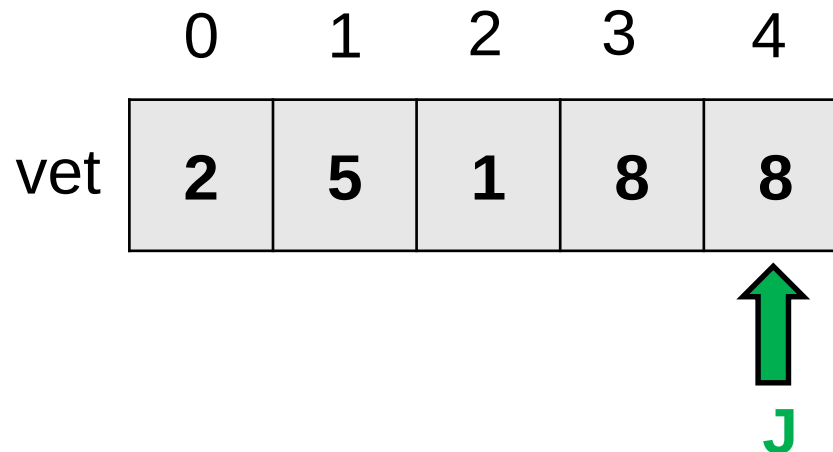
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	7
i	0
j	4



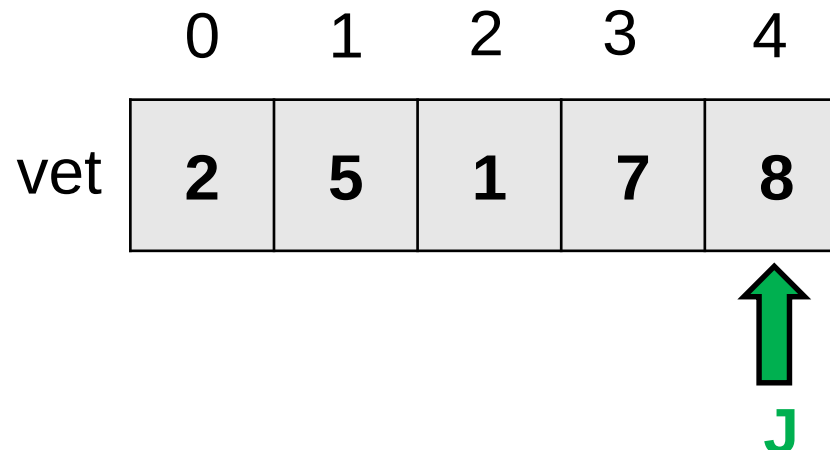
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	7
i	0
j	4



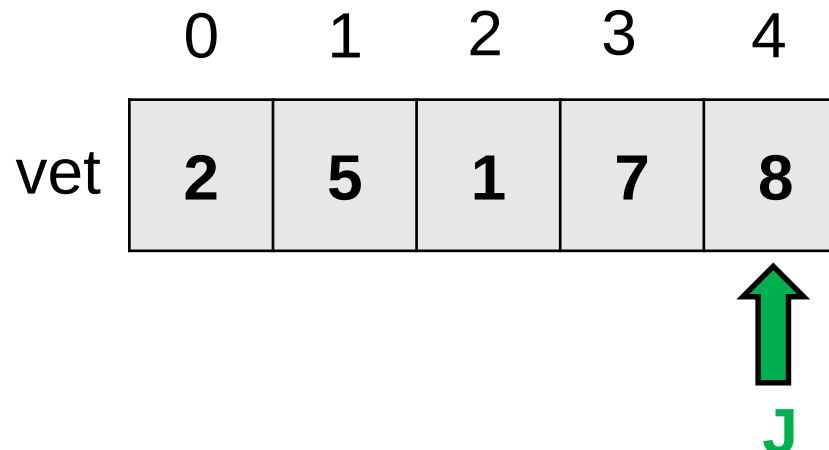
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	7
i	0
j	4



Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	7
i	0
j	5

	0	1	2	3	4
vet	2	5	1	7	8



Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	7
i	0
j	5

	0	1	2	3	4
vet	2	5	1	7	8



Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	7
i	1
j	5

	0	1	2	3	4
vet	2	5	1	7	8



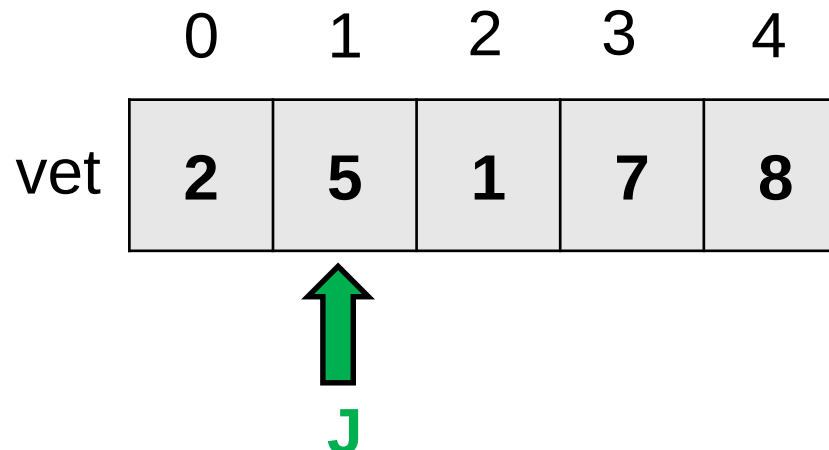
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
         for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	7
i	1
j	1



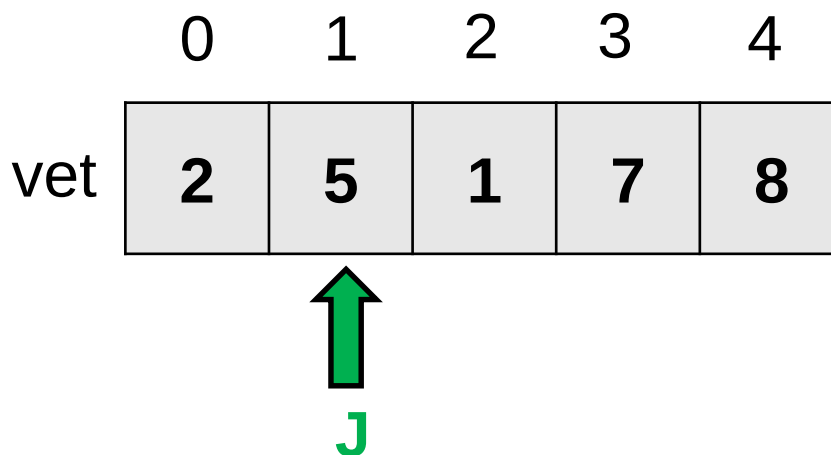
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	7
i	1
j	1



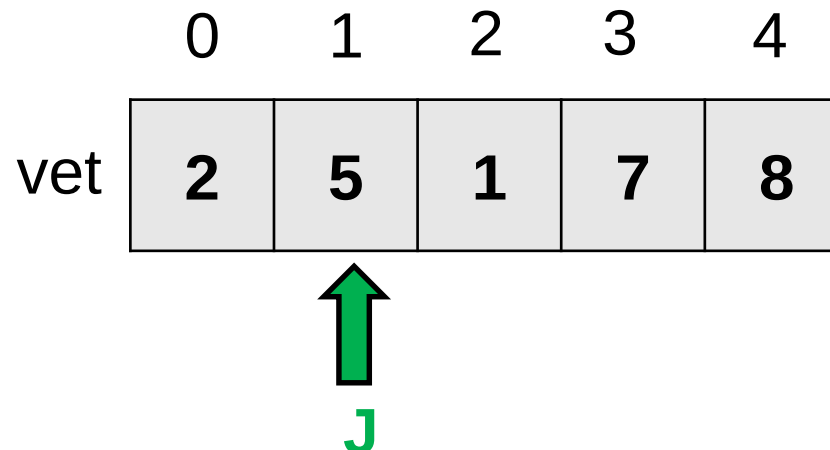
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
         for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	7
i	1
j	1



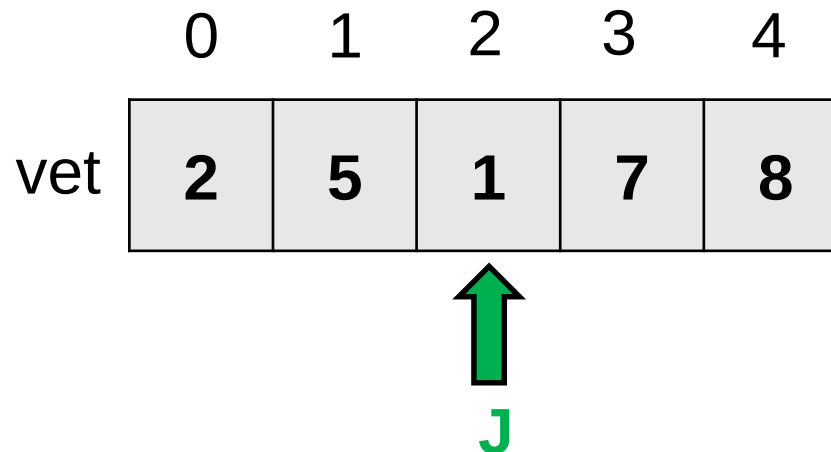
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
         for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	7
i	1
j	2



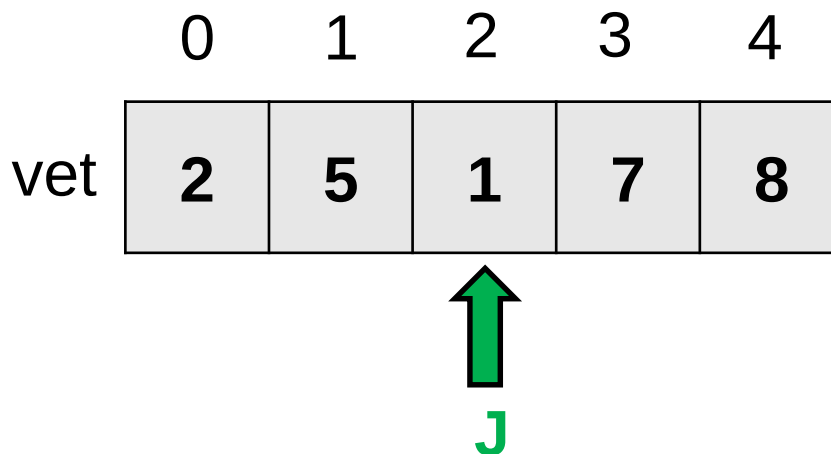
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	7
i	1
j	2



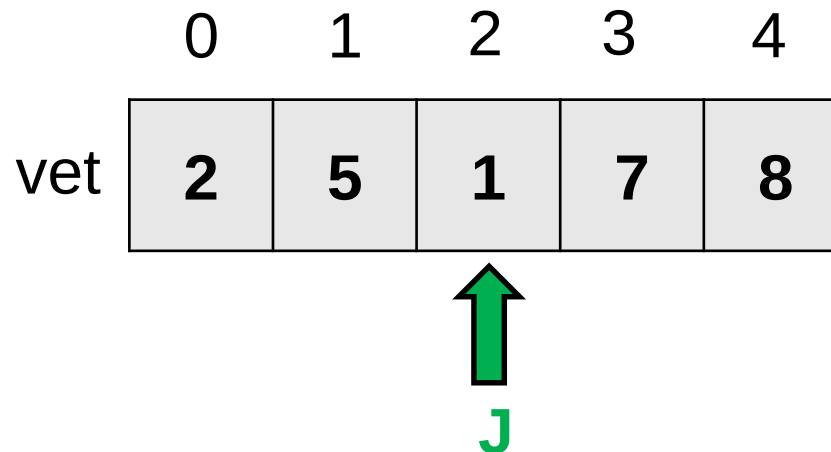
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	7
i	1
j	2



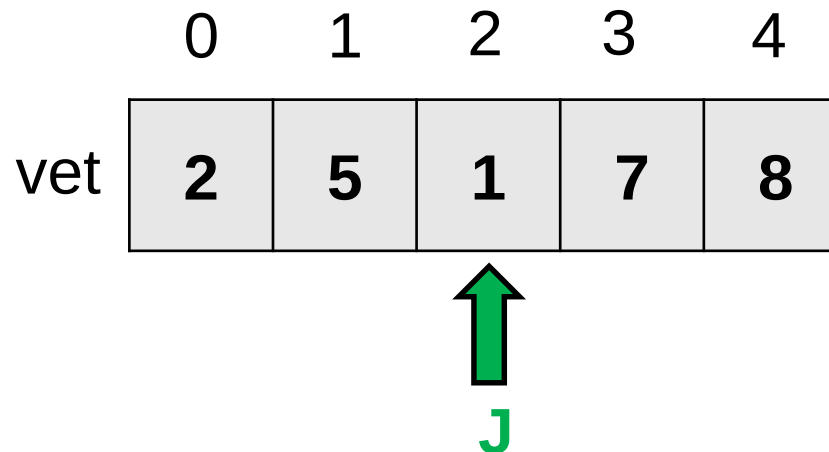
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	1
j	2



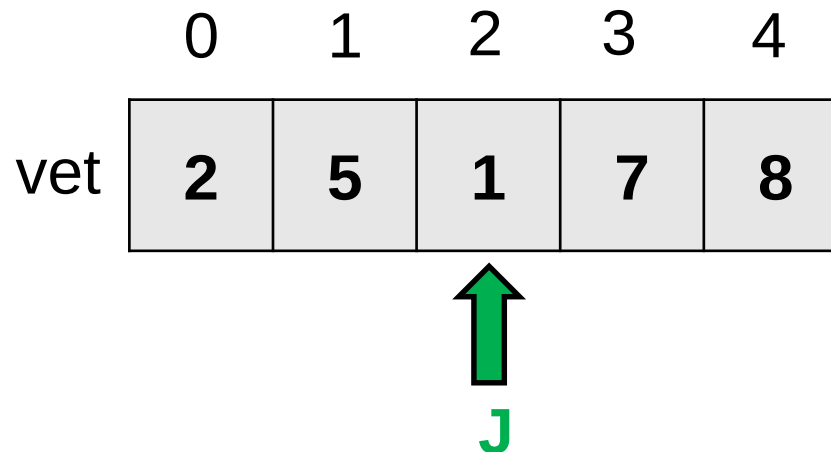
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	1
j	2



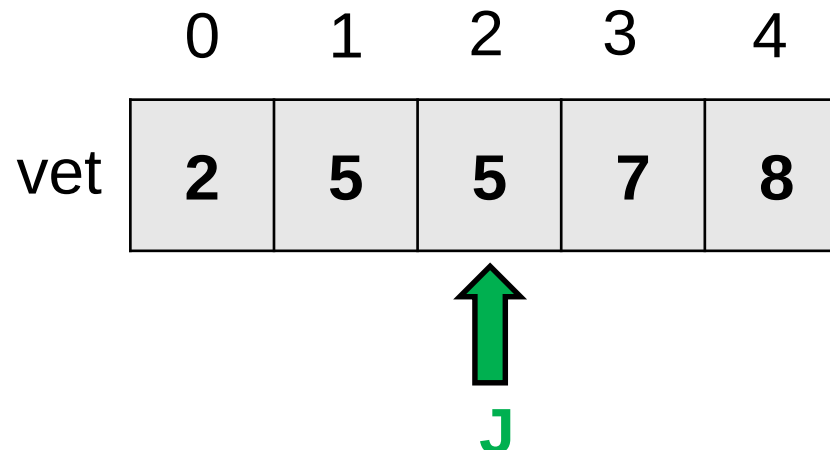
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	1
j	2



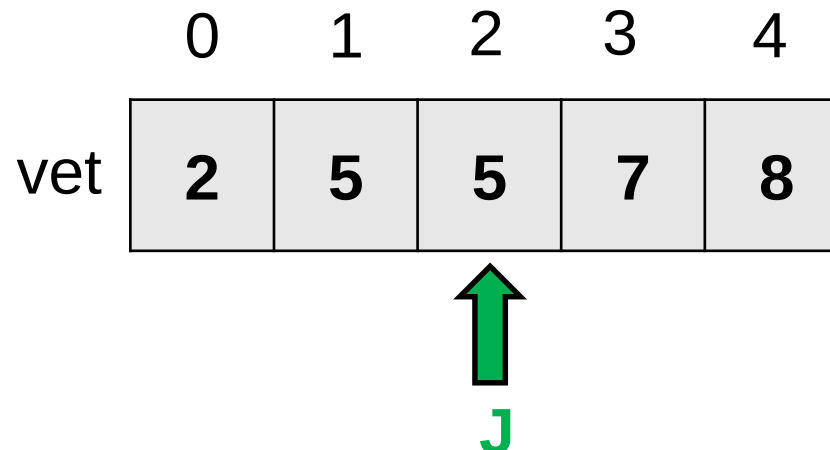
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	1
j	2



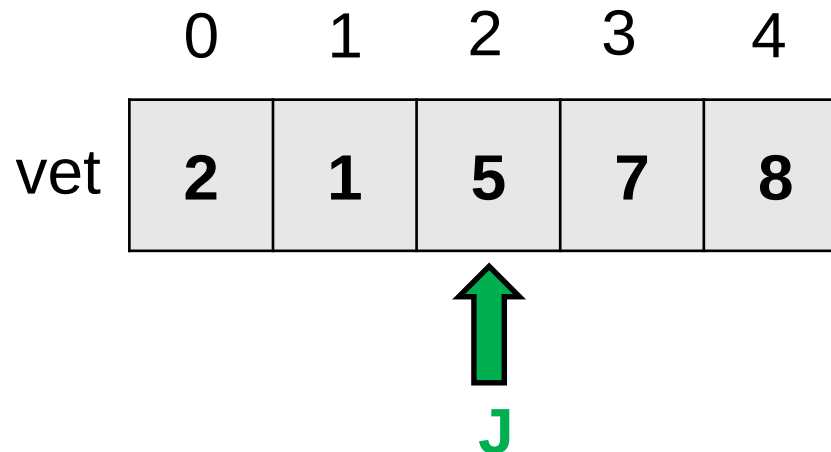
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	1
j	2



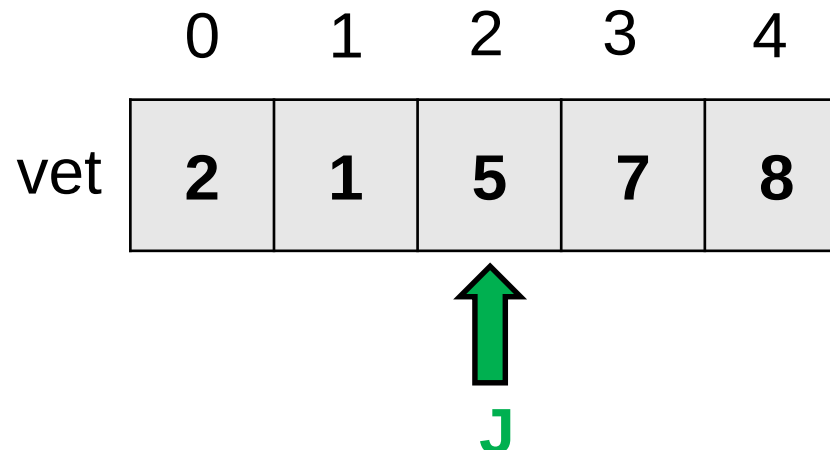
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
         for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	1
j	2



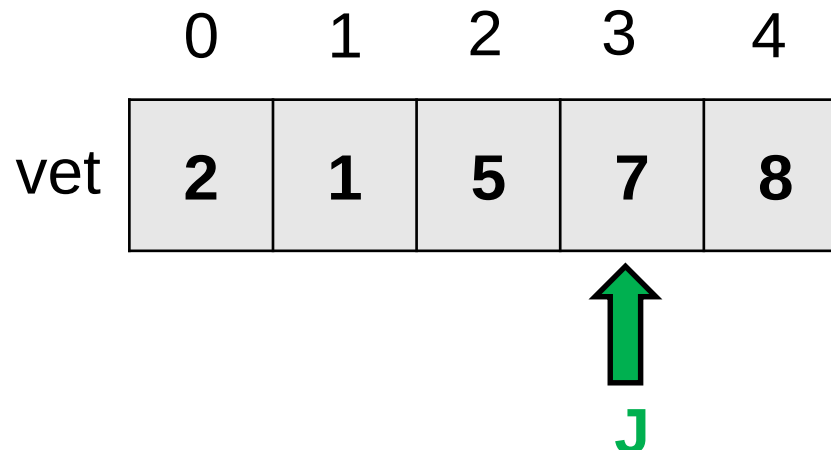
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
         for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	1
j	3



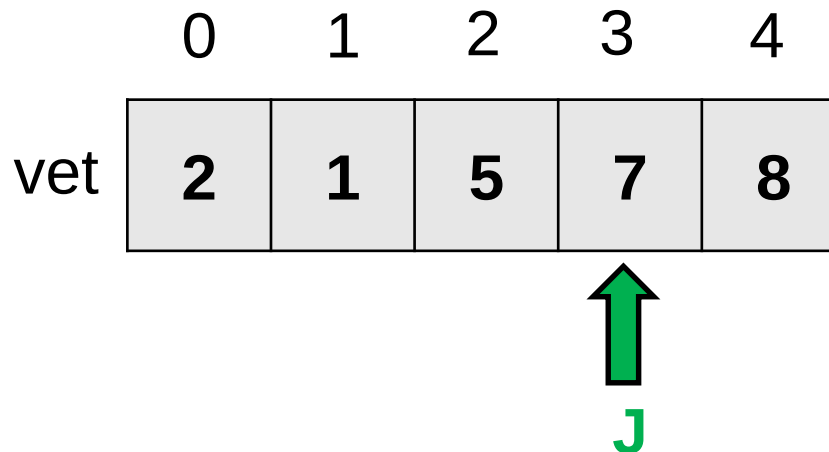
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	1
j	3



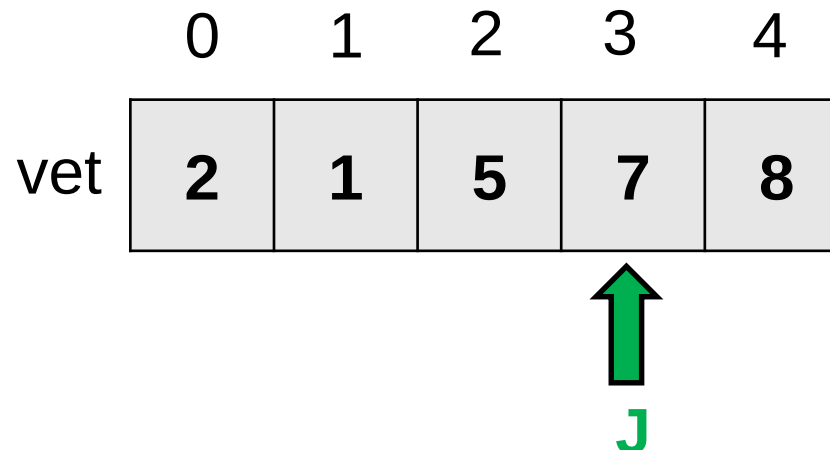
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
         for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	1
j	3



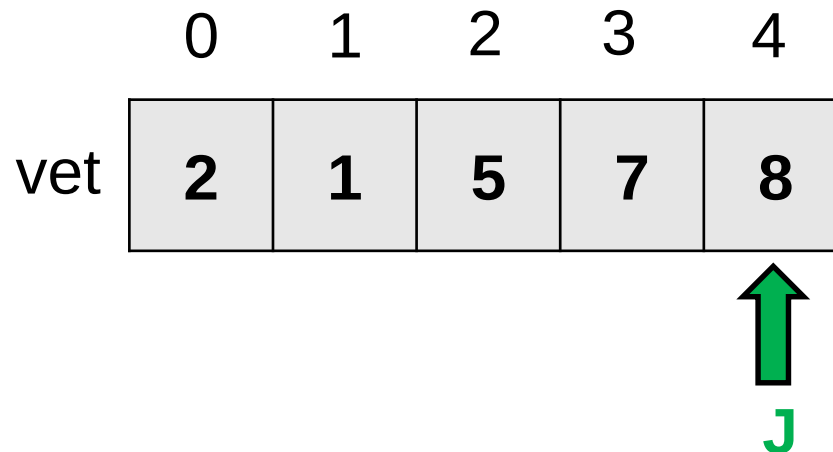
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	1
j	4



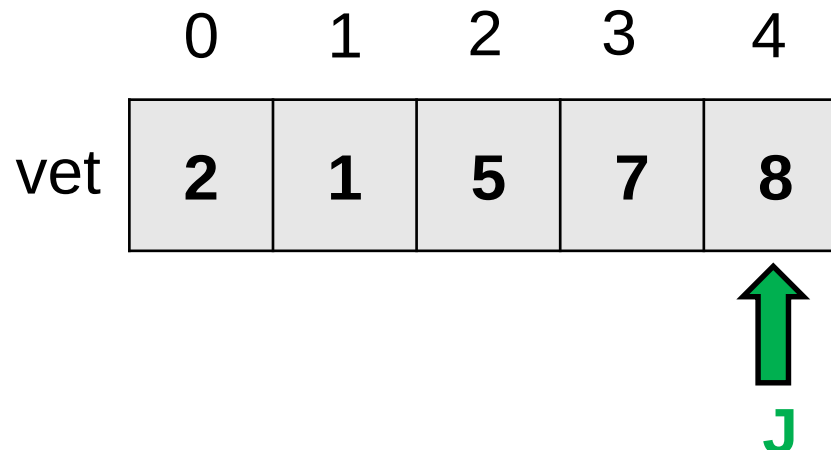
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	1
j	4



Bubble sort 2 – versão otimizada

```
#include <stdio.h>
```

```
#define tam 5
```

```
int main(){
```

```
    int vet[5] = {5, 2, 8, 1, 7};
```

```
    int i,j,aux;
```



```
    for(i = 0; i<tam-1; i++){
```

```
        for(j = 1; j < tam - i; j++){
```

```
            if(vet[j-1] > vet[j]){
```

```
                aux = vet[j];
```

```
                vet[j] = vet[j-1];
```

```
                vet[j-1] = aux;
```

```
            }
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

Variável	Conteúdo
aux	1
i	2
j	4

	0	1	2	3	4
vet	2	1	5	7	8



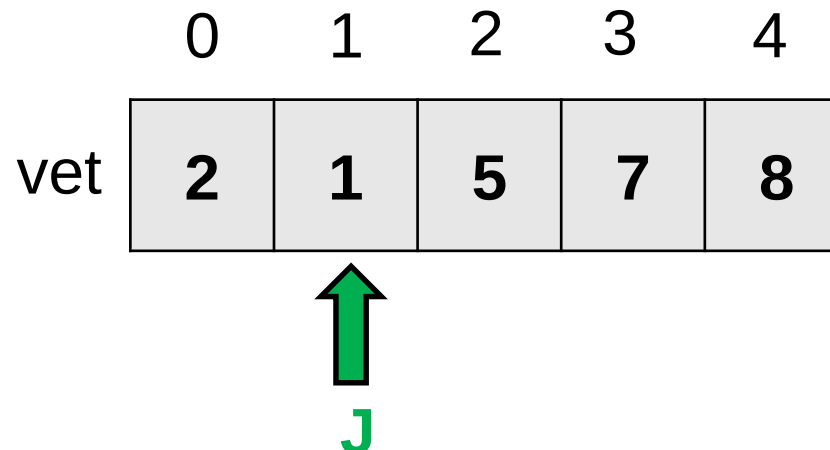
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
         for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	2
j	1



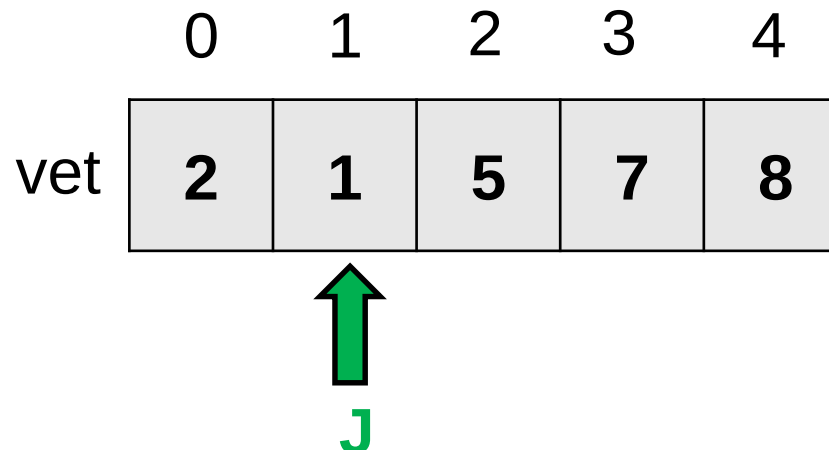
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	2
j	1



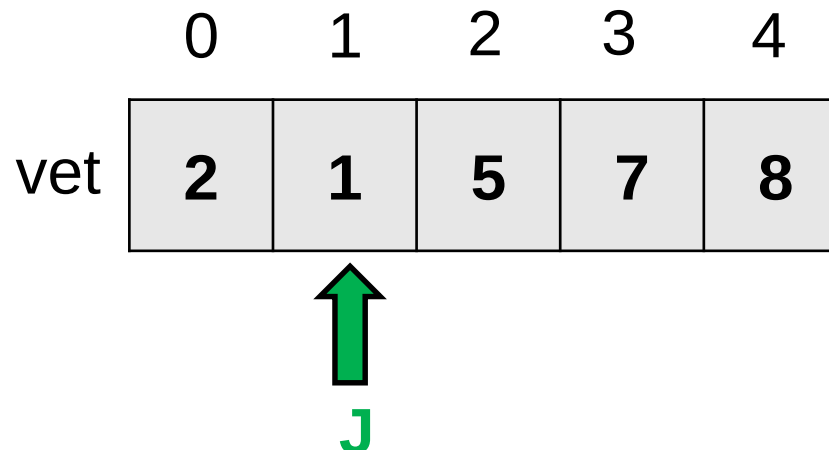
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	2
j	1



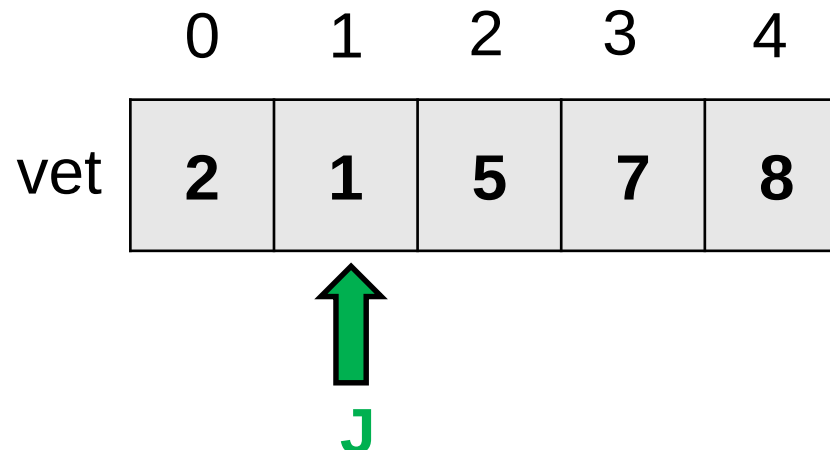
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	2
j	1



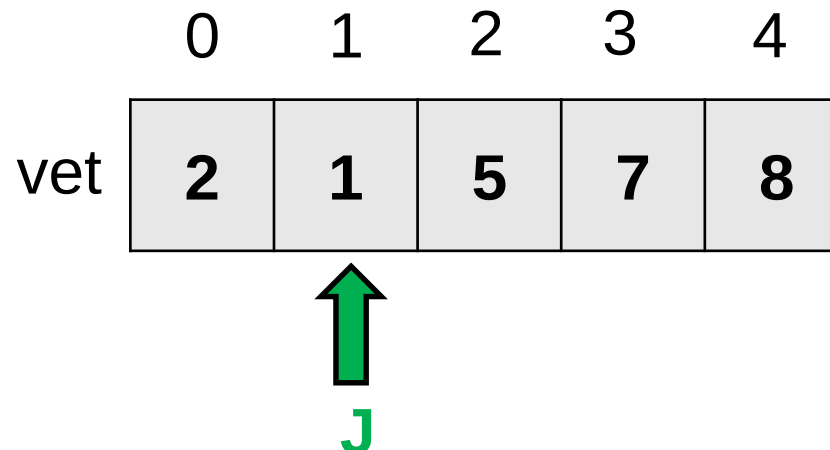
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	2
j	1



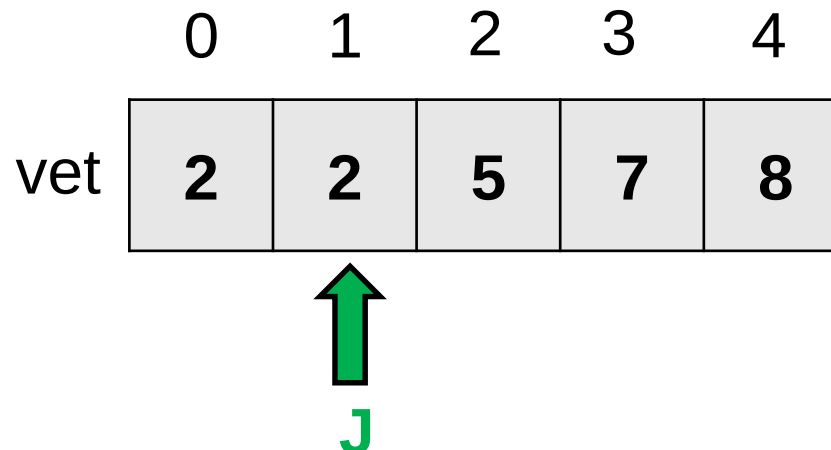
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	2
j	1



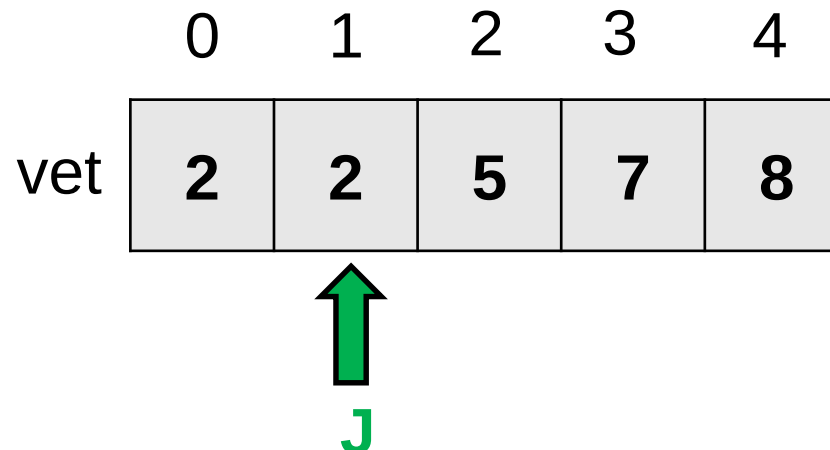
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	2
j	1



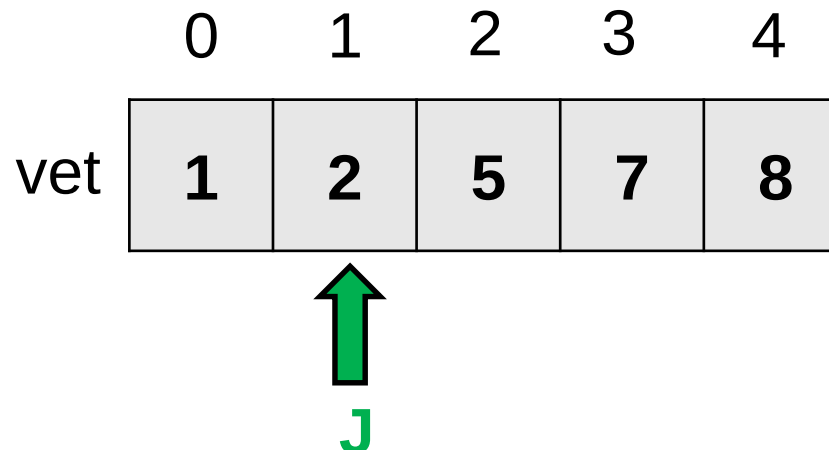
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	2
j	1



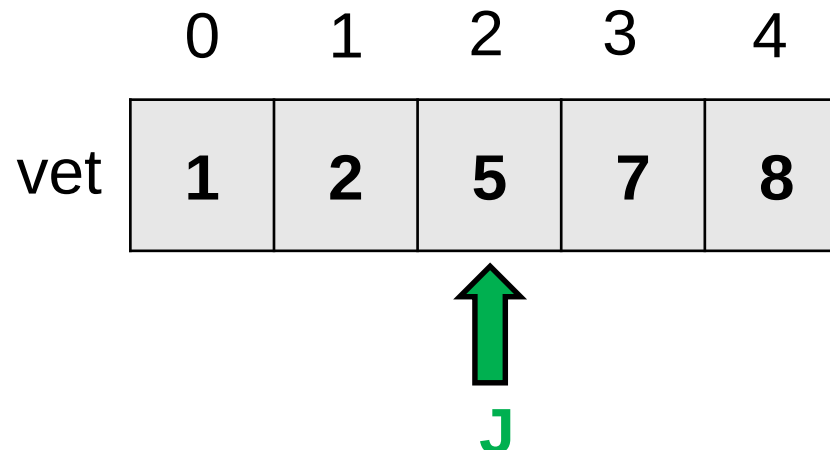
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
         for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	2
j	2



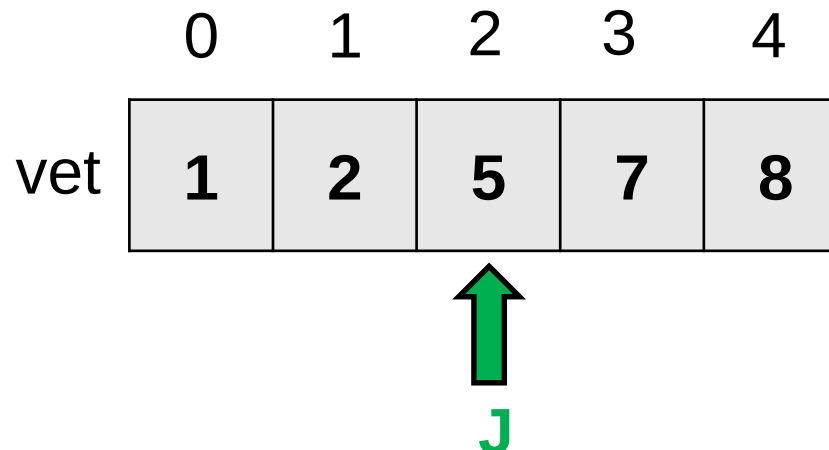
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	2
j	2



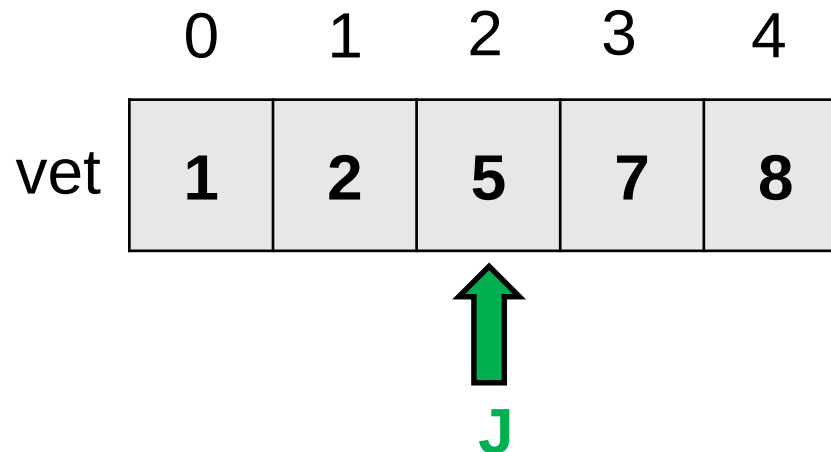
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	2
j	2



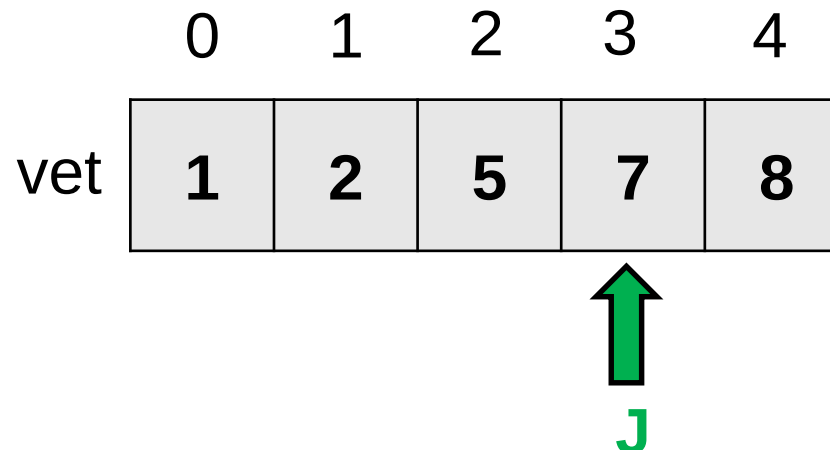
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	2
j	3



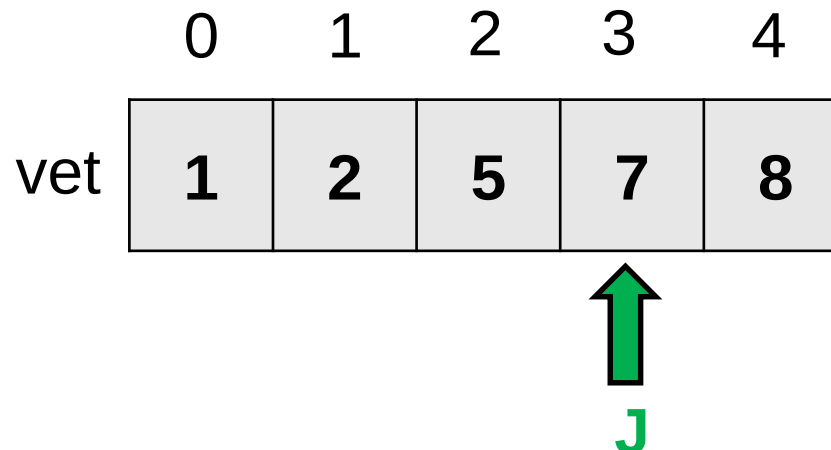
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	2
j	3



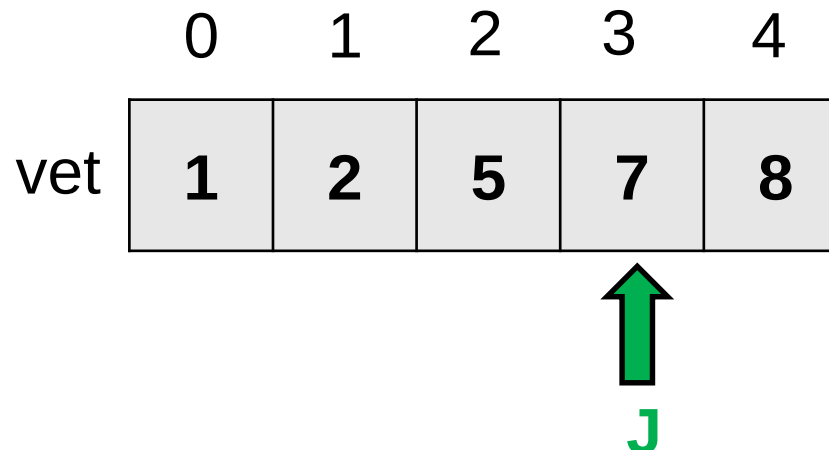
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	3
j	3



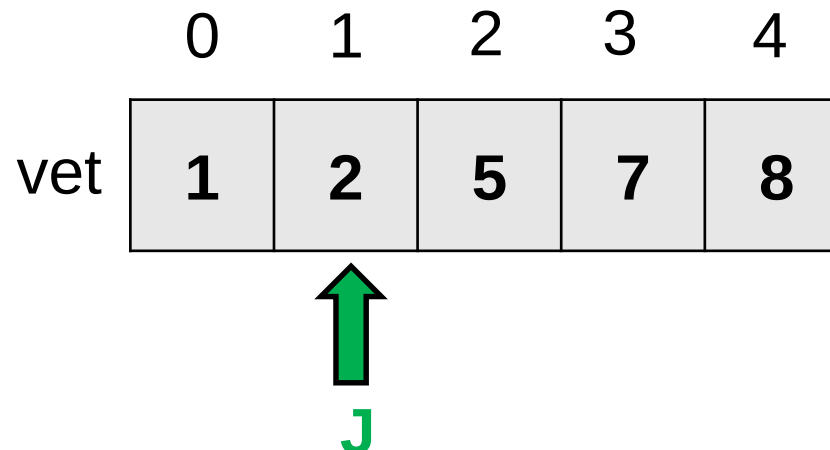
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
         for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	3
j	1



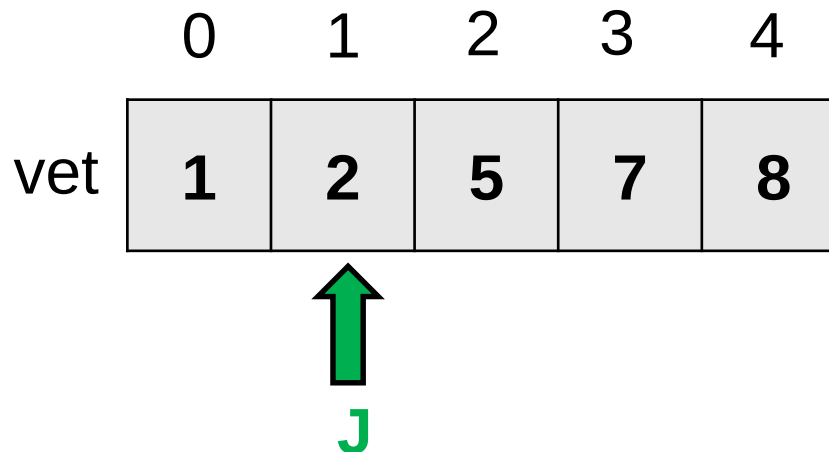
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	3
j	1



1 > 2 (F)

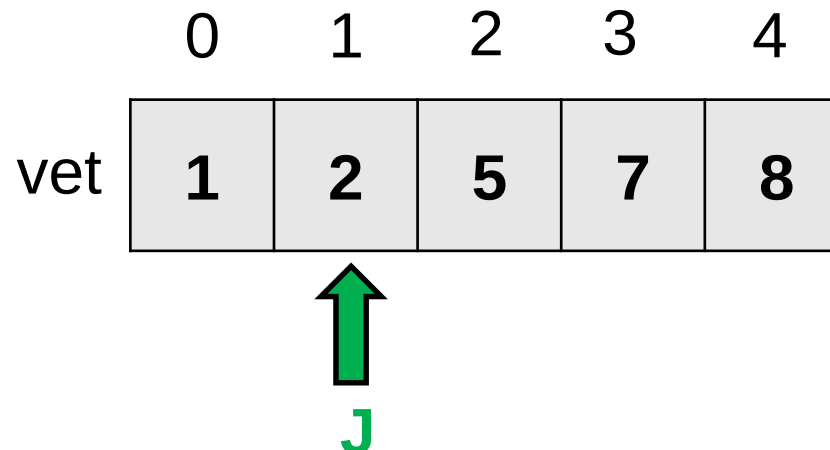
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
         for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	3
j	1



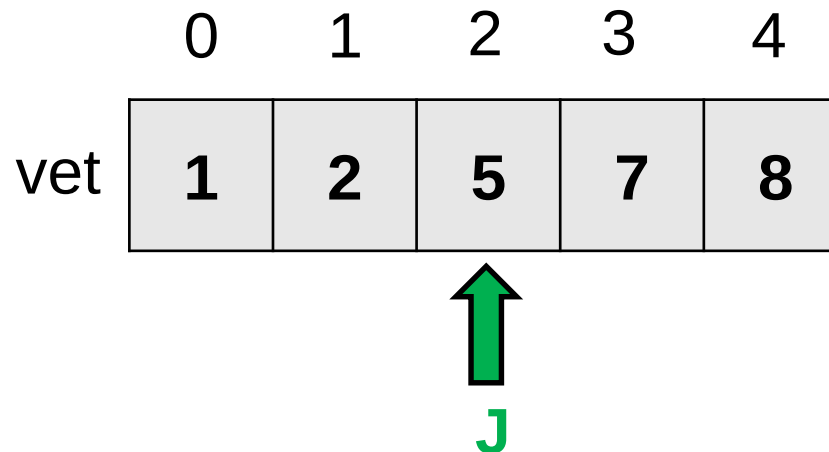
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
         for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	3
j	2



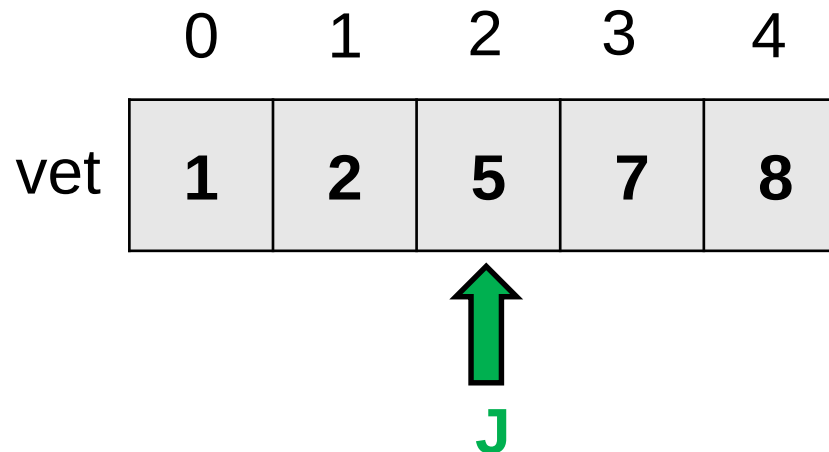
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	3
j	2



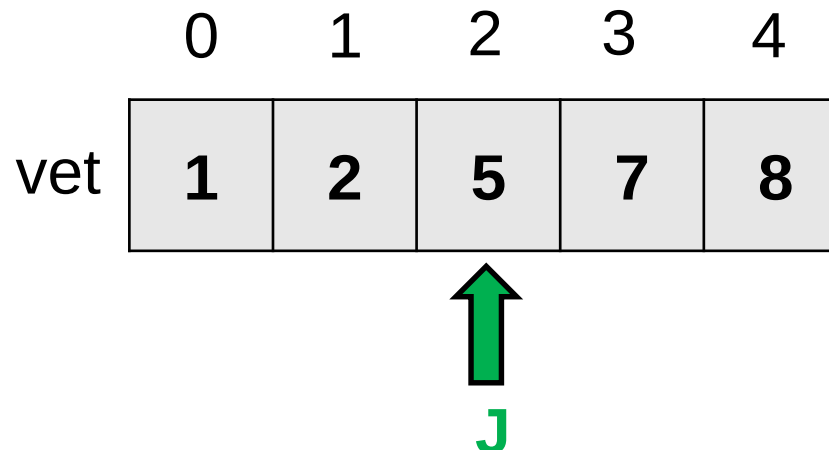
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	4
j	2



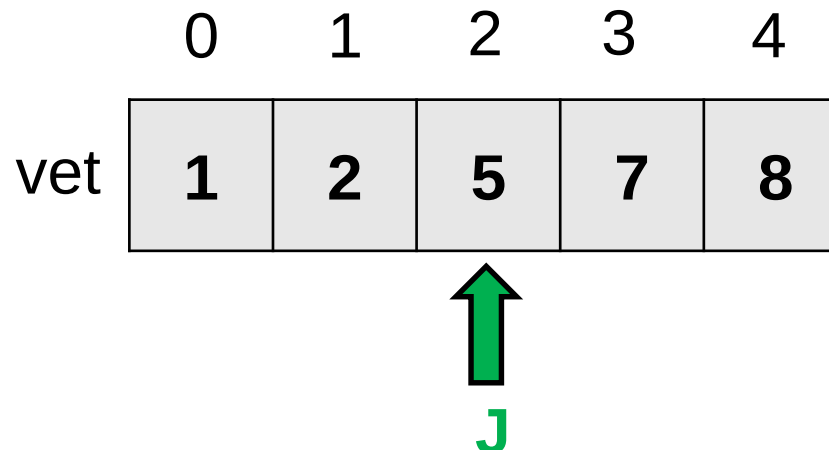
Bubble sort 2 – versão otimizada

```
#include <stdio.h>
#define tam 5

int main(){
    int vet[5] = {5, 2, 8, 1, 7};
    int i,j,aux;

    for(i = 0; i<tam-1; i++){
        for(j = 1; j < tam - i; j++){
            if(vet[j-1] > vet[j]){
                aux = vet[j];
                vet[j] = vet[j-1];
                vet[j-1] = aux;
            }
        }
    }
    return 0;
}
```

Variável	Conteúdo
aux	1
i	4
j	2



Exercício

- Faça um programa em linguagem C para ordenar de maneira decrescente um vetor de números inteiros de tamanho 10 utilizando o método do Bubble sort. Apresente na tela o vetor original e o vetor ordenado.